

ui.interactive_image 全面详细阐述

`ui.interactive_image` 是 NiceGUI 框架中专为**图像交互场景**设计的高级组件，基于 `ui.image` 扩展，核心新增了鼠标事件（点击、拖拽、缩放、平移）的精准坐标识别能力，支持像素级交互、图像标注、区域选择等复杂需求。该组件底层仍依赖 Quasar QImg，但强化了事件处理与坐标转换逻辑，是图像编辑、地图标注、数据可视化等场景的核心工具。以下从核心特性、使用场景、配置选项、事件系统、进阶用法等方面展开详细说明。

一、核心概述

1. 功能定位

- 基础能力：继承 `ui.image` 的所有特性（多源图像加载、样式配置、叠加层嵌套）；
- 核心增强：提供**图像坐标系与屏幕坐标系的自动转换**，支持获取鼠标在图像上的精准像素坐标；
- 交互支持：原生支持点击、双击、拖拽、滚轮缩放、平移等事件，且事件回调中直接返回图像像素坐标；
- 典型场景：图像标注（点、线、矩形）、区域选择、像素级交互（如点击图像某个位置触发操作）、缩放平移查看细节。

2. 核心优势

优势点	说明
坐标精准转换	自动处理图像缩放、拉伸后的坐标映射，直接返回原始图像的像素坐标（无需手动计算）；
丰富交互事件	内置 <code>click</code> 、 <code>dblclick</code> 、 <code>mousedown</code> 、 <code>mouseup</code> 、 <code>mousemove</code> 、 <code>wheel</code> 等事件，且事件参数统一包含像素坐标；
支持缩放平移	内置 <code>zoom</code> 、 <code>pan</code> 方法，可通过代码控制图像缩放比例与偏移量；
兼容叠加层	支持嵌套 <code>ui.label</code> 、 <code>ui.html</code> 等元素作为叠加层，可结合交互事件动态更新标注；
轻量化集成	无需额外依赖，API 设计与 <code>ui.image</code> 一致，学习成本低。

3. 与 ui.image 的核心差异

特性	ui.image	ui.interactive_image
坐标获取	仅支持屏幕坐标（相对组件的位置），需手动转换图像像素坐标；	自动返回图像像素坐标（基于原始图像尺寸），无需手动计算；
交互事件	基础鼠标事件（无坐标映射）；	增强型鼠标事件（含像素坐标、缩放比例、偏移量等参数）；
缩放平移	需手动实现；	内置缩放平移方法与事件支持；

特性	ui.image	ui.interactive_image
典型场景	静态图像展示、简单点击跳转；	图像标注、区域选择、像素级交互；

二、基础使用与图像源支持

`ui.interactive_image` 完全继承 `ui.image` 的图像源加载能力，支持 URL、本地文件、Base64、PIL 图像等所有格式，用法与 `ui.image` 一致。

基础示例（加载图像并获取点击坐标）

```
from nicegui import ui

# 加载网络图像（原始尺寸 640x360）
interactive_img = ui.interactive_image('https://picsum.photos/id/29/640/360')

# 绑定点击事件，获取像素坐标
@interactive_img.on('click')
def on_click(e):
    # e 为事件对象，包含图像像素坐标（x, y）、屏幕坐标（client_x, client_y）、缩放比例（scale）等
    ui.notify(f'点击图像像素坐标: ({e.x:.1f}, {e.y:.1f})')

ui.run()
```

多图像源示例

```
from nicegui import ui
from PIL import Image
import numpy as np

# 1. 本地文件
ui.interactive_image('static/logo.png').classes('w-48')

# 2. Base64 字符串
base64_str = ''
ui.interactive_image(base64_str).classes('w-16')

# 3. PIL 动态图像
random_pil_img = Image.fromarray(np.random.randint(0, 255, (200, 200, 3), dtype=np.uint8))
ui.interactive_image(random_pil_img).classes('w-32')

ui.run()
```

三、核心配置选项

`ui.interactive_image` 继承 `ui.image` 的所有配置属性（如 `classes`、`props`、`style`、`visible` 等），并新增了以下专属配置：

属性名	类型	说明	默认值
zoomable	bool	是否允许通过鼠标滚轮缩放图像	True
pannable	bool	是否允许拖拽平移图像（需先缩放，平移范围为图像边界）	True
min_scale	float	最小缩放比例（相对于原始图像尺寸）	0.1
max_scale	float	最大缩放比例	10.0
scale_step	float	滚轮每滚动一次的缩放步长（如 0.1 表示每次缩放 10%）	0.1
initial_scale	float	初始缩放比例	1.0
initial_offset	Tuple[float, float]	初始偏移量 (x, y)，单位为图像像素	(0, 0)
keep_aspect_ratio	bool	是否保持图像宽高比（缩放时不会拉伸）	True

配置示例（自定义缩放平移规则）

```
from nicegui import ui

# 配置不可缩放、可平移的交互式图像
ui.interactive_image(
    source='https://picsum.photos/id/30/800/600',
    zoomable=False, # 禁用滚轮缩放
    pannable=True,  # 启用拖拽平移
    initial_scale=0.8, # 初始缩放 80%
    keep_aspect_ratio=True, # 保持宽高比
).classes('w-full h-[400px] border-2 border-gray-300')

ui.run()
```

四、核心事件系统（重点）

`ui.interactive_image` 的核心价值在于**增强型事件处理**，所有鼠标事件都会返回图像像素坐标、缩放比例、偏移量等关键信息，无需手动转换。以下是支持的事件及参数说明：

1. 事件类型与参数

事件名	触发时机	事件参数 (e) 关键属性
click	鼠标左键点击图像	<code>x/y</code> ：图像像素坐标； <code>client_x/client_y</code> ：屏幕坐标； <code>scale</code> ：当前缩放比例； <code>offset</code> ：当前偏移量 (x, y)
dblclick	鼠标左键双击图像	同 <code>click</code>

事件名	触发时机	事件参数 (e) 关键属性
<code>mousedown</code>	鼠标按下（任意按键）	同 <code>click</code> + <code>button</code> ：鼠标按键（0 = 左键，1 = 中键，2 = 右键）
<code>mouseup</code>	鼠标松开（任意按键）	同 <code>mousedown</code>
<code>mousemove</code>	鼠标在图像上移动	同 <code>click</code> （实时返回坐标，支持拖拽跟踪）
<code>wheel</code>	鼠标滚轮滚动（缩放）	同 <code>click</code> + <code>delta_y</code> ：滚轮滚动方向（正 = 向上，负 = 向下）； <code>scale</code> ：缩放后的比例
<code>pan</code>	拖拽平移图像时触发（持续事件）	<code>delta_x/delta_y</code> ：平移偏移量（像素）； <code>offset</code> ：平移后的总偏移量； <code>scale</code> ：当前缩放比例
<code>zoom</code>	缩放图像时触发（滚轮或代码调用）	<code>scale</code> ：缩放后的比例； <code>delta</code> ：缩放变化量； <code>center</code> ：缩放中心点（图像像素坐标）
<code>load</code>	图像加载完成后触发	无额外参数（继承自 <code>ui.image</code> ）
<code>error</code>	图像加载失败时触发	无额外参数（继承自 <code>ui.image</code> ）

2. 事件绑定示例（核心场景）

示例 1：点击图像标注点（像素级定位）

```
from nicegui import ui

# 存储标注点
marks = []

# 创建交互式图像
img = ui.interactive_image('https://picsum.photos/id/31/800/600').classes('w-full')

# 点击图像添加红色标注点（叠加层）
@img.on('click')
def add_mark(e):
    # 记录标注点坐标
    marks.append((e.x, e.y))
    # 在点击位置添加红色圆点（绝对定位，基于图像容器）
    with img:
        ui.label('●').classes(
            'absolute text-red-500 text-2xl',
            f'translate-x-[-50%] translate-y-[-50%]', # 居中对齐点击位置
            f'left-[{e.client_x}px] top-[{e.client_y}px]' # 屏幕坐标定位（避免缩放偏移）
        ).tooltip(f'坐标: ({e.x:.0f}, {e.y:.0f})')

# 显示标注点列表
ui.label(f'标注点: {marks}').bind_text_from(lambda: f'标注点: {[f"({x:.0f}, {y:.0f})" for x, y in marks]}')
```

```
ui.run()
```

示例 2：拖拽绘制矩形（区域选择）

```
from nicegui import ui

# 存储矩形绘制状态
drawing = False
start_x = start_y = end_x = end_y = 0
rect_element = None

# 创建交互式图像
img = ui.interactive_image('https://picsum.photos/id/32/800/600').classes('w-full border-2 border-gray-300')

# 鼠标按下：开始绘制
@img.on('mousedown')
def start_drawing(e):
    global drawing, start_x, start_y, end_x, end_y, rect_element
    if e.button != 0: # 仅响应左键
        return
    drawing = True
    # 记录起始像素坐标
    start_x, start_y = e.x, e.y
    end_x, end_y = e.x, e.y
    # 创建矩形叠加层（透明边框）
    with img:
        rect_element = ui.html('''
            <div style="position: absolute; border: 2px solid red; background:
transparent;"></div>
            ''', sanitize=False)

# 鼠标移动：更新矩形尺寸
@img.on('mousemove')
def update_drawing(e):
    global end_x, end_y
    if not drawing or rect_element is None:
        return
    # 记录当前像素坐标
    end_x, end_y = e.x, e.y
    # 计算矩形屏幕坐标（基于图像容器的相对位置）
    # 注意：需通过图像的缩放比例和偏移量转换为屏幕坐标
    scale = e.scale
    offset_x, offset_y = e.offset
    # 原始图像坐标 -> 屏幕坐标（相对 img 组件）
    screen_start_x = (start_x - offset_x) * scale
    screen_start_y = (start_y - offset_y) * scale
    screen_end_x = (end_x - offset_x) * scale
    screen_end_y = (end_y - offset_y) * scale
    # 计算矩形位置和尺寸（确保左上角为起点）
    left = min(screen_start_x, screen_end_x)
    top = min(screen_start_y, screen_end_y)
```

```

width = abs(screen_end_x - screen_start_x)
height = abs(screen_end_y - screen_start_y)
# 更新矩形样式
rect_element.style(f'left: {left}px; top: {top}px; width: {width}px; height: {height}px;')

# 鼠标松开：结束绘制
@img.on('mouseup')
def finish_drawing(e):
    global drawing
    if not drawing:
        return
    drawing = False
# 显示矩形区域信息（原始图像像素坐标）
rect_info = f'矩形区域: ({start_x:.0f}, {start_y:.0f}) → ({end_x:.0f}, {end_y:.0f})'
ui.notify(rect_info)

ui.run()

```

示例 3：滚轮缩放与平移事件监听

```

from nicegui import ui

# 创建交互式图像
img = ui.interactive_image('https://picsum.photos/id/33/800/600').classes('w-full h-[400px]')

# 显示当前缩放比例和偏移量
status_label = ui.label('缩放比例: 1.0 | 偏移量: (0, 0)')

# 监听缩放事件
@img.on('zoom')
def on_zoom(e):
    status_label.set_text(f'缩放比例: {e.scale:.2f} | 偏移量: {e.offset[0]:.0f}, {e.offset[1]:.0f}')

# 监听平移事件
@img.on('pan')
def on_pan(e):
    status_label.set_text(f'缩放比例: {e.scale:.2f} | 偏移量: {e.offset[0]:.0f}, {e.offset[1]:.0f}')

# 按钮控制：重置缩放和平移
ui.button('重置', on_click=lambda: img.reset())

ui.run()

```

五、核心方法（控制与操作）

`ui.interactive_image` 提供一系列方法用于手动控制图像的缩放、平移、状态重置等，支持代码驱动交互：

方法名	作用	参数说明
<code>zoom(scale, center=None)</code>	手动设置缩放比例	<code>scale</code> : 目标缩放比例; <code>center</code> : 缩放中心点 (图像像素坐标, 默认图像中心)
<code>pan(delta_x, delta_y)</code>	手动平移图像	<code>delta_x / delta_y</code> : 平移偏移量 (像素, 基于当前缩放比例)
<code>set_offset(x, y)</code>	手动设置图像偏移量	<code>x / y</code> : 目标偏移量 (像素)
<code>reset()</code>	重置图像到初始状态 (缩放比例 = 1.0, 偏移量 =(0,0))	无参数
<code>get_scale()</code>	获取当前缩放比例	返回 <code>float</code> 类型
<code>get_offset()</code>	获取当前偏移量	返回 <code>Tuple[float, float]</code> (x,y)
<code>get_image_size()</code>	获取原始图像的尺寸 (宽、高)	返回 <code>Tuple[int, int]</code> (width, height)
<code>force_reload()</code>	强制刷新图像 (继承自 <code>ui.image</code>)	无参数
<code>set_source(source)</code>	切换图像源 (继承自 <code>ui.image</code>)	<code>source</code> : 新图像源 (URL / 本地路径 / Base64/PIL 图像)

方法使用示例（代码控制交互）

```
from nicegui import ui

img = ui.interactive_image('https://picsum.photos/id/34/800/600').classes('w-full h-[400px]')

# 按钮控制缩放
ui.row([
    ui.button('放大', on_click=lambda: img.zoom(img.get_scale() + 0.2)),
    ui.button('缩小', on_click=lambda: img.zoom(max(0.1, img.get_scale() - 0.2))),
    ui.button('重置', on_click=img.reset),
])

# 按钮控制平移
ui.row([
    ui.button('左移', on_click=lambda: img.pan(-50, 0)),
    ui.button('右移', on_click=lambda: img.pan(50, 0)),
    ui.button('上移', on_click=lambda: img.pan(0, -50)),
    ui.button('下移', on_click=lambda: img.pan(0, 50)),
])

# 显示图像信息
ui.label().bind_text_from(
    lambda: f'原始尺寸: {img.get_image_size()[0]}x{img.get_image_size()[1]} | 缩放比例: {img.get_scale():.2f} | 偏移量: {img.get_offset()[0]:.0f}, {img.get_offset()[1]:.0f}'
```

```
)
```

```
ui.run()
```

六、进阶用法（核心场景落地）

场景 1：图像标注工具（点、线、矩形）

结合叠加层和事件，实现完整的图像标注功能：

```
from nicegui import ui

class ImageAnnotator:
    def __init__(self, image_source):
        self.image_source = image_source
        self.mode = 'point' # 标注模式: point/line/rect
        self.annotations = [] # 存储标注数据
        self.temp_annotation = None # 临时标注（绘制中）
        self.ui = self._build_ui()

    def _build_ui(self):
        # 主容器
        with ui.card().classes('w-full'):
            # 标注模式选择
            with ui.row().classes('mb-2'):
                ui.radio(['point', 'line', 'rect'], value='point', on_change=lambda e:
setattr(self, 'mode', e.value)).props('inline')
                ui.button('清除所有', on_click=self.clear_annotations)

            # 交互式图像
            self.img = ui.interactive_image(self.image_source).classes('w-full h-[500px]')

            # 绑定事件
            self.img.on('mousedown', self._on_mousedown)
            self.img.on('mousemove', self._on_mousemove)
            self.img.on('mouseup', self._on_mouseup)

        return self.img

    def _on_mousedown(self, e):
        if e.button != 0:
            return
        # 记录起始坐标（图像像素坐标）
        start_pos = (e.x, e.y)
        if self.mode == 'point':
            # 点标注：直接添加
            self._add_point_annotation(start_pos)
        elif self.mode == 'line':
            # 线标注：初始化临时线
            self.temp_annotation = {'type': 'line', 'start': start_pos, 'end': start_pos,
'element': None}
            self._draw_temp_annotation(e)
```



```

elif self.mode == 'rect':
    # 矩形标注: 初始化临时矩形
    self.temp_annotation = {'type': 'rect', 'start': start_pos, 'end': start_pos,
'element': None}
    self._draw_temp_annotation(e)

def _on_mousemove(self, e):
    if self.temp_annotation is None:
        return
    # 更新临时标注的结束坐标
    self.temp_annotation['end'] = (e.x, e.y)
    self._draw_temp_annotation(e)

def _on_mouseup(self, e):
    if self.temp_annotation is None:
        return
    # 保存标注并清除临时元素
    self.annotations.append(self.temp_annotation)
    self.temp_annotation['element'] = None
    self.temp_annotation = None

def _add_point_annotation(self, pos):
    # 添加点标注 (红色圆点)
    with self.img:
        ui.label('●').classes(
            'absolute text-red-500 text-2xl translate-x-[-50%] translate-y-[-50%]',
            f'left-{self._image_to_screen_x(pos[0])}px',
            f'top-{self._image_to_screen_y(pos[1])}px'
        )
    self.annotations.append({'type': 'point', 'pos': pos})

def _draw_temp_annotation(self, e):
    # 移除之前的临时元素
    if self.temp_annotation['element'] is not None:
        self.temp_annotation['element'].delete()

    scale = e.scale
    offset_x, offset_y = e.offset
    start_x, start_y = self.temp_annotation['start']
    end_x, end_y = self.temp_annotation['end']

    # 转换为屏幕坐标 (相对 img 组件)
    screen_start_x = (start_x - offset_x) * scale
    screen_start_y = (start_y - offset_y) * scale
    screen_end_x = (end_x - offset_x) * scale
    screen_end_y = (end_y - offset_y) * scale

    if self.temp_annotation['type'] == 'line':
        # 绘制临时线
        with self.img:
            self.temp_annotation['element'] = ui.html(f'''
                <svg style="position: absolute; top: 0; left: 0; width: 100%; height:
100%;">

```

```

        <line x1="{screen_start_x}" y1="{screen_start_y}" x2="{
{screen_end_x}" y2="{screen_end_y}"
        stroke="blue" stroke-width="2" />

    </svg>
    ''' , sanitize=False)
elif self.temp_annotation['type'] == 'rect':
    # 绘制临时矩形
    left = min(screen_start_x, screen_end_x)
    top = min(screen_start_y, screen_end_y)
    width = abs(screen_end_x - screen_start_x)
    height = abs(screen_end_y - screen_start_y)
    with self.img:
        self.temp_annotation['element'] = ui.html(f'''
            <div style="position: absolute; left: {left}px; top: {top}px; width:
{width}px; height: {height}px;
            border: 2px solid green; background: transparent;"></div>
        ''', sanitize=False)

def _image_to_screen_x(self, x):
    # 图像X坐标 -> 屏幕X坐标（简化版，适用于未缩放偏移时）
    scale = self.img.get_scale()
    offset_x = self.img.get_offset()[0]
    return (x - offset_x) * scale

def _image_to_screen_y(self, y):
    # 图像Y坐标 -> 屏幕Y坐标（简化版）
    scale = self.img.get_scale()
    offset_y = self.img.get_offset()[1]
    return (y - offset_y) * scale

def clear_annotations(self):
    # 清除所有标注（遍历子元素并删除）
    for child in self.img.children:
        child.delete()
    self.annotations.clear()

# 初始化标注工具
annotator = ImageAnnotator('https://picsum.photos/id/35/1200/800')

ui.run()

```

场景 2：像素级交互（点击图像获取像素颜色）

结合 PIL 图像库，实现点击图像获取对应像素的 RGB 颜色：

```

from nicegui import ui
from PIL import Image
import requests
from io import BytesIO

class PixelColorPicker:
    def __init__(self, image_url):

```

```

self.image_url = image_url
self.pil_image = self._load_pil_image()
self.ui = self._build_ui()

def _load_pil_image(self):
    # 从URL加载PIL图像
    response = requests.get(self.image_url)
    return Image.open(BytesIO(response.content)).convert('RGB')

def _build_ui(self):
    with ui.card().classes('w-full'):
        self.img = ui.interactive_image(self.image_url).classes('w-full h-[400px]')
        self.color_label = ui.label('点击图像获取颜色').classes('mt-2')

        # 绑定点击事件
        self.img.on('click', self._on_image_click)

    return self.img

def _on_image_click(self, e):
    # 获取图像像素坐标（确保在图像范围内）
    img_width, img_height = self.pil_image.size
    x = max(0, min(int(e.x), img_width - 1))
    y = max(0, min(int(e.y), img_height - 1))

    # 获取RGB颜色
    r, g, b = self.pil_image.getpixel((x, y))
    color_hex = f'#{r:02x}{g:02x}{b:02x}'

    # 更新显示
    self.color_label.set_text(f'像素坐标: ({x}, {y}) | RGB: ({r}, {g}, {b}) | 十六进制: {color_hex}')
    # 显示颜色块
    self.color_label.style(f'background-color: {color_hex}; color: white; padding: 4px; border-radius: 4px;')

# 初始化颜色拾取器
picker = PixelColorPicker('https://picsum.photos/id/36/800/600')

ui.run()

```

场景 3：结合 Lottie 动画的交互式标注

在交互式图像上添加动态动画作为标注，提升交互体验：

```

from nicegui import ui

# 引入 Lottie 播放器脚本
ui.add_body_html('<script src="https://unpkg.com/@lottiefiles/lottie-player@latest/dist/lottie-player.js"></script>')

```

```
img = ui.interactive_image('https://picsum.photos/id/37/800/600').classes('w-full h-[400px]')

# 点击图像添加动态标注 (Lottie 动画)
@img.on('click')
def add_lottie_annotation(e):
    with img:
        # Lottie 动画 (标记点动画)
        ui.html(f'''
            <lottie-player
                src="https://assets1.lottiefiles.com/packages/lf20_9zqk0f.json"
                loop autoplay
                style="position: absolute; left: {e.client_x - 20}px; top: {e.client_y - 20}px; width: 40px; height: 40px;"
            />
            ''', sanitize=False)

ui.run()
```

七、注意事项与优化建议

1. 坐标转换细节

- 事件参数中的 `x/y` 是**原始图像的像素坐标** (不受缩放、偏移影响), 直接对应图像文件的像素位置;
- `client_x/client_y` 是**相对于 `ui.interactive_image` 组件的屏幕坐标** (受缩放、偏移影响), 用于叠加层定位;
- 若需手动转换坐标, 可使用公式: `屏幕坐标 = (图像坐标 - 偏移量) * 缩放比例`。

2. 性能优化

- 避免在 `mousemove` 事件中执行复杂计算 (如频繁创建 / 删除元素), 可通过节流 (throttle) 优化;
- 当图像尺寸较大 (如超过 2000x2000 像素) 时, 建议先压缩图像, 避免缩放平移时卡顿;
- 大量标注时, 可使用 Canvas 替代 DOM 元素作为叠加层 (减少 DOM 节点数量)。

3. 兼容性问题

- `ui.interactive_image` 依赖现代浏览器的鼠标事件 API, 不支持 IE 等老旧浏览器;
- 若图像加载失败, 事件将无法触发, 建议添加 `error` 事件处理:

```
@img.on('error')
def on_image_error():
    ui.notify('图像加载失败, 请检查URL', type='error')
```

4. 安全注意事项

- 若加载外部图像，需确保图像来源可信（避免 XSS 风险）；
- 使用 `ui.html` 嵌入自定义内容时，`sanitize=False` 可能存在安全风险，需过滤不可信 HTML 代码。

5. 边界处理

- 平移时，图像默认会被限制在组件边界内（无法平移出可视区域），无需手动处理；
- 缩放时，需注意 `min_scale` 和 `max_scale` 的设置，避免过度缩放导致图像失真或性能下降。

总结

`ui.interactive_image` 是 NiceGUI 中面向复杂图像交互场景的核心组件，通过**精准坐标识别**、**丰富事件支持**、**代码可控的缩放平移**三大核心能力，完美解决了 `ui.image` 在交互场景中的局限性。其继承了 `ui.image` 的所有基础特性，同时新增了针对图像交互的增强功能，可广泛应用于图像标注、区域选择、像素级交互、地图交互等场景。

掌握该组件的关键在于：

1. 理解 `图像坐标` 与 `屏幕坐标` 的区别与转换逻辑；
2. 熟练运用各类事件参数（`x/y/scale/offset`）实现精准交互；
3. 结合叠加层（DOM/SVG/Lottie）实现可视化标注。